アルゴリズム特論

第5章　高階関数

学籍番号　氏名

今日の流れ（110分）

時間	内容	手を動かす
0:00–0:05	導入・第4章の振り返り	H0 準備
0:05–0:20	5.1 カリー化関数	H1
0:20–0:50	5.2 高階実演 / 5.3 道具箱 / 5.4 ラムダ式	
0:50–1:00	**前半のまとめ演習**（課題 問1〜3）	10分
1:00–1:05	休憩	
1:05–1:30	5.5 畳み込み / 5.6 `$`	H4
1:30–1:42	5.7 関数合成	H5
1:42–1:50	まとめ・課題の説明	

> 前半は「関数を**作る・渡す**」、後半は「関数で**まとめる・つなぐ**」

目次

5.1　カリー化関数

5.2　高階実演

5.3　関数プログラマの道具箱

5.4　ラムダ式

5.5　畳み込み、見込みアリ！

5.6　`$` を使った関数適用

5.7　関数合成

第4章の振り返り：再帰

```haskell
maximum' :: (Ord a) => [a] -> a
maximum' [] = error "empty list"
maximum' [x] = x                        -- 基底部
maximum' (x:xs) = max x (maximum' xs) -- 再帰部
```

```haskell
reverse' :: [a] -> [a]
reverse' [] = []                        -- 基底部
reverse' (x:xs) = reverse' xs ++ [x]
```

- **基底部**：再帰せずに答えが決まる場合
- **再帰部**：小さな問題に分けて自分自身を呼ぶ
- リストは `[]` と `x:xs` に分けて考える

> 今日：この「リストを先頭から1つずつ処理する」パターンを**高階関数**で短く書く

HANDS-ON H0　準備：ファイルを作って GHCi で読み込む

0:03–0:05

1. devcontainer を起動し、ターミナルで作業ディレクトリへ移動

```
$ cd lectures/section05
$ ghci report5.hs          # 課題ファイルを読み込んで起動
```

2. GHCi でよく使うコマンド

コマンド	意味
`:l ファイル名`	ファイルを読み込む（load）
`:r`	編集したファイルを再読み込み（reload）
`:t 式`	式の型を表示（type）
`:q`	GHCi を終了

3. 課題のテストを一括で実行：`$ runghc report5.hs`（未実装は `[--]` と表示される）

目次

5.1　カリー化関数

5.2　高階実演

5.3　関数プログラマの道具箱

5.4　ラムダ式

5.5　畳み込み、見込みアリ！

5.6　`$` を使った関数適用

5.7　関数合成

5.1　カリー化関数

- **Haskell のすべての関数は引数を1つしかとらない**

```
max 4 5      -- 2つを比べて大きい方
(+) 4 5      -- 2つを足す
(++) [1] [2] -- 2つのリストを結合
```

複数の引数をとるように見えるが、実際は
カリー化された関数

- **カリー化**：複数の引数をとる関数を「引数を1つ受け取り、関数を返す関数」の連なりとして扱うこと
- **高階関数**：関数を引数に取ったり、返り値として返したりする関数

5.1　カリー化関数

- `max 4 5` は `(max 4) 5` と同じ

```
ghci> max 4 5
5
ghci> (max 4) 5
5
ghci> :t max
max :: Ord a => a -> a -> a
```

1. `max 4` が「4 と比べて大きい方を返す関数」を返す
2. その関数を `5` に適用する

`a -> a -> a` は `a -> (a -> a)` と同じ

部分適用　関数を本来より少ない引数で呼び出すと、残りの引数をとる関数が得られる

5.1　カリー化関数

- 部分適用の例：`multThree`

```
multThree :: Int -> Int -> Int -> Int   -- = Int -> (Int -> (Int -> Int))
multThree x y z = x * y * z
```

`multThree 3 5 9` は `((multThree 3) 5) 9` と同じ（スペース区切りの関数適用は左結合）

式	得られるもの
`multThree 3`	「3 と残り2つの引数を掛ける関数」
`(multThree 3) 5`	「3 と 5 と残り1つの引数を掛ける関数」
`((multThree 3) 5) 9`	`135`

> 関数は「材料を受け取って何かを作る小さな工場」。
> 3 を入れると、数ではなく「ちょっと小さくなった工場」が出てくる

5.1　カリー化関数

- 部分適用で新しい関数を作る

```
ghci> let multTwoWithNine = multThree 9
ghci> multTwoWithNine 2 3
54
```

```
compareWithHundred :: Int -> Ordering
compareWithHundred x = compare 100 x
-- ↓ 両辺の右端の x を省略できる
compareWithHundred = compare 100
```

```
ghci> compareWithHundred 99
GT
```

`multThree 9` は 2 引数の関数
→ 残り2つを掛けて 9 倍

`compare 100` は
「数を受け取り 100 と比較する関数」

型 `(Ord a) => a -> (a -> Ordering)`
に
100 を与えたので、型宣言は同じまま

5.1　カリー化関数

- セクション：中置関数の部分適用

```haskell
divideByTen :: (Floating a) => a -> a
divideByTen = (/10)

ghci> divideByTen 200   -- 200 / 10 と同じ
20.0
```

```haskell
isUpperAlphanum :: Char -> Bool
isUpperAlphanum = (`elem` ['A'..'Z'])
```

片側だけに値を置いて**括弧で囲む**と、
値を置かなかった側を引数にとる関数になる

`(/10) 200` = `200 / 10`

`(10/) 200` = `10 / 200`

> **注意**：`(-4)` は「マイナス4」の意味。4 を引く関数は `(subtract 4)` と書く

5.1　カリー化関数

- **関数そのものは表示できない**

```
ghci> multThree 3 4
<interactive>:1:1: error:
    • No instance for (Show (Int -> Int)) arising from a use of ‘print’
        (maybe you haven't applied a function to enough arguments?)
```

- `multThree 3 4` は `Int -> Int` 型の**関数**を返す
- 関数は `Show` 型クラスのインスタンスではないので、文字列にできない
- `1 + 1` なら結果の `2` に `show` を呼んで `"2"` を表示できる

> カリー化と部分適用の働きは、この章の土台。ここだけは必ず押さえる

HANDS-ON H1　部分適用とセクションを試す

3分

`ghci handson.hs` で起動し、次を入力して結果を予想してから確かめる

```
ghci> :t max
ghci> :t max 4                -- 型はどう変わる？
ghci> (max 4) 5
ghci> multTwoWithNine 2 3
ghci> (/10) 200
ghci> (10/) 200               -- 左右を入れ替えると？
ghci> (-4)                    -- 関数ではなく数になる
ghci> (subtract 4) 10
ghci> multThree 3 4           -- エラーになる理由は？
```

目次

5.1　カリー化関数

5.2　高階実演

5.3　関数プログラマの道具箱

5.4　ラムダ式

5.5　畳み込み、見込みアリ！

5.6　`$` を使った関数適用

5.7　関数合成

5.2　高階実演

- 関数を受け取り、2回適用する `applyTwice`

```
applyTwice :: (a -> a) -> a -> a
applyTwice f x = f (f x)
```

```
ghci> applyTwice (+3) 10
16
ghci> applyTwice (++ " HAHA") "HEY"
"HEY HAHA HAHA"
ghci> applyTwice ("HAHA " ++) "HEY"
"HAHA HAHA HEY"
ghci> applyTwice (multThree 2 2) 9
144
ghci> applyTwice (3:) [1]
[3,3,1]
```

型の**括弧は必須**

第1引数が「`a` を受け取り `a` を返す関数」であることを表す

`->` は右結合なので、括弧がないと `a -> a -> a -> a`（3引数関数）になる

> 1引数の関数を渡したいときは、部分適用で作って渡せばよい

5.2　高階実演

- `zipWith` を実装する

```
zipWith' :: (a -> b -> c) -> [a] -> [b] -> [c]
zipWith' _ [] _ = []
zipWith' _ _ [] = []
zipWith' f (x:xs) (y:ys)
    = f x y : zipWith' f xs ys
```

- 第1引数：2引数の関数 `a -> b -> c`
- 第2・3引数：`[a]` と `[b]`
- 結果：`[c]`

基底部はどちらかが空リスト

→ `zip` と同じ形。`(x, y)` の代わりに `f x y` を返す

> 型がよく分からなくなったら、型宣言を省略して `:t` で推論させる

5.2 高階実演

- `zipWith'` の使用例

```
ghci> zipWith' (+) [4,2,5,6] [2,6,2,3]
[6,8,7,9]
ghci> zipWith' max [6,3,2,1] [7,3,1,5]
[7,3,2,5]
ghci> zipWith' (++) ["foo ", "bar ", "baz "] ["fighters", "hoppers", "aldrin"]
["foo fighters","bar hoppers","baz aldrin"]
ghci> zipWith' (*) (replicate 5 2) [1..]
[2,4,6,8,10]
ghci> zipWith' (zipWith' (*)) [[1,2,3],[3,5,6],[2,3,4]] [[3,2,2],[3,4,5],[5,4,3]]
[[3,4,6],[9,20,30],[10,12,12]]
```

> 1つの高階関数を、渡す関数を変えるだけで多くの用途に使える

5.2 高階実演

- `flip` を実装する：最初の2つの引数を入れ替える

```haskell
flip' :: (a -> b -> c) -> (b -> a -> c)
flip' f = g
    where g x y = f y x
```

```haskell
-- カリー化を使って簡潔に
flip' :: (a -> b -> c) -> b -> a -> c
flip' f y x = f x y
```

```
ghci> zip [1,2,3,4,5] "hello"
[(1,'h'),(2,'e'),(3,'l'),(4,'l'),(5,'o')]
ghci> flip' zip [1,2,3,4,5] "hello"
[('h',1),('e',2),('l',3),('l',4),('o',5)]
ghci> zipWith (flip' div) [2,2..] [10,8,6,4,2]     -- flip' div 2 10 = div 10 2
[5,4,3,2,1]
```

目次

5.1　カリー化関数

5.2　高階実演

5.3　関数プログラマの道具箱

5.4　ラムダ式

5.5　畳み込み、見込みアリ！

5.6　`$` を使った関数適用

5.7　関数合成

5.3　関数プログラマの道具箱

- `map`：リストのすべての要素に関数を適用

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (x:xs) = f x : map f xs
```

```
ghci> map (+3) [1,5,3,1,6]
[4,8,6,4,9]
ghci> map (++ "!") ["BIFF", "BANG", "POW"]
["BIFF!","BANG!","POW!"]
ghci> map (replicate 3) [3..6]
[[3,3,3],[4,4,4],[5,5,5],[6,6,6]]
ghci> map (map (^2)) [[1,2],[3,4,5,6],[7,8]]
[[1,4],[9,16,25,36],[49,64]]
ghci> map fst [(1,2),(3,5),(6,3),(2,6),(2,5)]
[1,3,6,2,2]
```

`a` から `b` への関数と `a` のリストを受け取り、`b` のリストを返す

`map (+3) [1,5,3,1,6]` は `[x+3 | x <- [1,5,3,1,6]]` と同じ

> `map` の `map` のような場面では、`map` のほうが読みやすい

5.3　関数プログラマの道具箱

- `filter`：述語を満たす要素だけを残す

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p (x:xs)
    | p x       = x : filter p xs
    | otherwise = filter p xs
```

述語：`True` か `False` を返す関数

`p x` が `True` なら新しいリストに含め、`False` なら含めない

```
ghci> filter (>3) [1,5,3,2,1,6,4,3,2,1]
[5,6,4]
ghci> filter even [1..10]
[2,4,6,8,10]
ghci> filter (`elem` ['a'..'z']) "u LaUgH aT mE BeCaUsE I aM diFfeRent"
"uagameasadifeent"
```

5.3　関数プログラマの道具箱

- `map`・`filter` とリスト内包表記

```
ghci> filter (<15) (filter even [1..20])
[2,4,6,8,10,12,14]
ghci> [x | x <- [1..20], x < 15, even x]
[2,4,6,8,10,12,14]
```

どちらを使うかにルールはない。
読みやすいほうを選ぶ

第4章のクイックソートを `filter` で

```
quicksort :: (Ord a) => [a] -> [a]
quicksort [] = []
quicksort (x:xs) =
    let smallerOrEqual = filter (<= x) xs
        larger = filter (> x) xs
    in  quicksort smallerOrEqual
          ++ [x] ++ quicksort larger
```

5.3　関数プログラマの道具箱

- **10万以下で 3829 で割り切れる最大の数**

```haskell
largestDivisible :: Integer
largestDivisible = head (filter p [100000,99999..])
    where p x = x `mod` 3829 == 0
```

```
ghci> largestDivisible
99554
```

- 降順のリストを `filter` するので、条件を満たす最大の数は**先頭**に現れる
- Haskell は**遅延評価**なので、最初の解が見つかった時点で評価が止まる
  → リストが有限でも無限でも関係ない

5.3　関数プログラマの道具箱

- `takeWhile`：条件を満たす間だけ取り出す

```
ghci> takeWhile (/=' ') "elephants know how to party"
"elephants"
```

10000 より小さい奇数の平方数の和

```
ghci> sum (takeWhile (<10000) (filter odd (map (^2) [1..])))
166650
ghci> sum (takeWhile (<10000) [m | m <- [n^2 | n <- [1..]], odd m])
166650
```

`[1..]` を `map (^2)` → `filter odd` → `takeWhile (<10000)` → `sum`

> 無限リストから始めて、map して filter して、必要な分だけ切り出して足すだけ

5.3　関数プログラマの道具箱

- コラッツ列

1. 任意の自然数から開始する
2. 数が 1 ならば終了
3. 偶数なら 2 で割る
4. 奇数なら 3 倍して 1 を足す
5. 新しい値で繰り返す

```
chain :: Integer -> [Integer]
chain 1 = [1]
chain n
    | even n = n : chain (n `div` 2)
    | odd n  = n : chain (n * 3 + 1)
```

```
ghci> chain 10
[10,5,16,8,4,2,1]
ghci> chain 1
[1]
ghci> chain 30
[30,15,46,23,70,35,106,53,
 160,80,40,20,10,5,16,8,4,2,1]
```

基底部は `1`（どの列も 1 で終わる）

問題：1〜100 の数のうち、長さが 15 より長いコラッツ列の開始数はいくつあるか

5.3　関数プログラマの道具箱

- **`numLongChains`**

```
numLongChains :: Int
numLongChains = length (filter isLong (map chain [1..100]))
    where isLong xs = length xs > 15
```

1. `[1..100]` を `chain` で `map` し、コラッツ列のリストを得る
2. 長さが 15 より長いかを調べる述語 `isLong` で `filter`
3. 残った列の個数を `length` で数える（結果は `66`）

※ 教科書の問題文は「長さ15以上」だがコードは `> 15`。本講義はコードに合わせ「15より長い」とする

※ `length` は `Int` を返す。一般の `Num a` にしたいなら `fromIntegral` を使う

5.3　関数プログラマの道具箱

- `map` に複数の引数をとる関数を与える

```
ghci> let listOfFuns = map (*) [0..]
ghci> (listOfFuns !! 4) 5
20
```

`(*)` は2引数関数なので、1つだけ与えると1引数関数になる

`map (*) [0..]`
= `[(0*), (1*), (2*), (3*), (4*), ...]`

4番目の要素 `(4*)` を取り出して 5 に適用 → `4 * 5` = `20`

> リストの要素が「関数」になることもある

目次

5.1　カリー化関数

5.2　高階実演

5.3　関数プログラマの道具箱

5.4　ラムダ式

5.5　畳み込み、見込みアリ！

5.6　`$` を使った関数適用

5.7　関数合成

5.4　ラムダ式

- **1回だけ必要な関数を作る無名関数**

```
numLongChains :: Int
numLongChains = length (filter (\xs -> length xs > 15)
                               (map chain [1..100]))
```

`\` + 引数 + `->` + 本体。普通は括弧で囲む

`\` はギリシャ文字のラムダ λ に見立てたもの

`where` で `isLong` を定義する代わりに、その場で関数を渡している

不要なラムダに注意

```
map (+3) [1,6,3,2]
map (\x -> x + 3) [1,6,3,2]
```

同じ結果なら部分適用のほうが読みやすい

5.4　ラムダ式

- 複数の引数とパターンマッチ

```
ghci> zipWith (\a b -> (a * 30 + 3) / b) [5,4,3,2,1] [1,2,3,4,5]
[153.0,61.5,31.0,15.75,6.6]
ghci> map (\(a,b) -> a + b) [(1,2),(3,5),(6,3),(2,6),(2,5)]
[3,8,9,8,7]
```

```
addThree' :: Int -> Int -> Int -> Int
addThree' = \x -> \y -> \z -> x + y + z

flip' :: (a -> b -> c) -> b -> a -> c
flip' f = \x y -> f y x
```

- パターンは1つしか書けない。マッチに失敗するとランタイムエラー
- `flip'` のラムダ版は「新しい関数を作る」という意図がよく表れる

EXERCISE 前半のまとめ演習（課題 問1〜3）

10分

`report5.hs` の `undefined` を書き換え、`:r` と `runghc report5.hs` で確かめる

問	内容	ヒント
問1	`zipWith'` を実装	基底部はどちらかが `[]`
問2	`flip'` を実装	`flip' f y x = ...`
問3	`chain` と `numLongChains`	`map` → `filter` → `length`

```
$ runghc report5.hs
[OK] 問1 zipWith'
[--] 問2 flip'  (未実装またはエラー)
...
```

早く終わった人：`numLongChains` をラムダ式で書き直す

休憩　5分

再開は 1:05 から

後半は「畳み込み」「`$`」「関数合成」

前半のまとめ演習：解答例

```haskell
zipWith' :: (a -> b -> c) -> [a] -> [b] -> [c]
zipWith' _ [] _ = []
zipWith' _ _ [] = []
zipWith' f (x:xs) (y:ys) = f x y : zipWith' f xs ys

flip' :: (a -> b -> c) -> b -> a -> c
flip' f y x = f x y

chain :: Integer -> [Integer]
chain 1 = [1]
chain n
    | even n = n : chain (n `div` 2)
    | odd n  = n : chain (n * 3 + 1)

numLongChains :: Int
numLongChains = length (filter (\xs -> length xs > 15) (map chain [1..100]))
```

目次

5.1　カリー化関数

5.2　高階実演

5.3　関数プログラマの道具箱

5.4　ラムダ式

5.5　畳み込み、見込みアリ！

5.6　`$` を使った関数適用

5.7　関数合成

5.5　畳み込み、見込みアリ！

- **畳み込み（fold）：リストを単一の値にまとめる**

第4章の再帰関数の多くは同じパターン

- 基底部は空リスト
- `x:xs` で先頭要素と残りに分解

畳み込み関数が受け取るもの

1. **2引数関数**（`+` や `div` など）
2. **アキュムレータ**（演算結果を一時保存する値）の初期値
3. 畳み込むリスト

> リストを1要素ずつ1回だけ走査して何かを返す関数は、畳み込みで書ける

5.5　畳み込み、見込みアリ！

- `foldl`：左畳み込み

```
foldl :: (a -> b -> a) -> a -> [b] -> a
```

```
sum' :: (Num a) => [a] -> a
sum' xs = foldl (\acc x -> acc + x) 0 xs
```

```
ghci> sum' [3,5,2,1]
11
```

acc	x	残り	新しい acc
0	3	[5,2,1]	3
3	5	[2,1]	8
8	2	[1]	10
10	1	[]	**11**

最初のアキュムレータとリストの先頭に2引数関数を適用 → 新しいアキュムレータ → 次の要素と…を繰り返す

※ 今の GHCi で `:t foldl` とすると `Foldable t => (b -> a -> b) -> b -> t a -> b` と表示される。`t a` を `[a]` と読み替えれば同じ意味

5.5　畳み込み、見込みアリ！

- カリー化で簡潔に

```
sum' :: (Num a) => [a] -> a
sum' xs = foldl (\acc x -> acc + x) 0 xs
```

↓ `\acc x -> acc + x` は `(+)` と同じ

```
sum' :: (Num a) => [a] -> a
sum' = foldl (+) 0
```

- `foldl (+) 0` はリストを受け取る関数を返すので、`xs` を省略できる
- 一般に `foo a = bar b a` は、カリー化のおかげで `foo = bar b` と書ける

5.5　畳み込み、見込みアリ！

- `foldr`：右畳み込み

```
foldl :: (a -> b -> a) -> a -> [b] -> a
foldr :: (a -> b -> b) -> b -> [a] -> b
```

```
map' :: (a -> b) -> [a] -> [b]
map' f xs = foldr (\x acc -> f x : acc) [] xs
```

```
map' f xs = foldl (\acc x -> acc ++ [f x]) [] xs
```

2引数関数の引数の順番が逆

- `foldl`：`\acc x -> ...`
- `foldr`：`\x acc -> ...`

`map' (+3) [1,2,3]`

右から `6:[]` → `5:[6]` → `4:[5,6]`

`++` は `:` よりはるかに遅いので、リストを作るときは普通 `foldr`

※ foldr もリストは左から走査する。結果が右から畳んだように見える

5.5　畳み込み、見込みアリ！

- `foldr` で `elem` を実装する

```haskell
elem' :: (Eq a) => a -> [a] -> Bool
elem' y ys = foldr (\x acc -> if x == y then True else acc) False ys
```

- アキュムレータは真理値（アキュムレータの型 = 結果の型）
- 初期値は `False`：空リストに対しても正しい結果になる
- 今の要素が探している値なら `True`、そうでなければアキュムレータをそのまま返す

> アキュムレータの値（結果の値）は、数・真理値・リストなど任意の型でよい

5.5　畳み込み、見込みアリ！

- `foldl1` と `foldr1`

```haskell
maximum' :: (Ord a) => [a] -> a
maximum' = foldl1 max
```

- 初期アキュムレータを与えなくてよい
- リストの先頭（または末尾）を初期アキュムレータとして使う

- 要素をアキュムレータと `max` で比べ、大きいほうを新しいアキュムレータにする
- 空リストに対して呼ぶとランタイムエラー（`foldl`・`foldr` なら空リストでも動く）

> 空リストに意味がない関数なら `foldl1` / `foldr1` が使える

5.5　畳み込み、見込みアリ！

- **畳み込みの例**

```haskell
reverse' :: [a] -> [a]
reverse' = foldl (\acc x -> x : acc) []
-- 引数が逆なだけの (:) なので
reverse' = foldl (flip (:)) []

product' :: (Num a) => [a] -> a
product' = foldl (*) 1
```

```haskell
filter' :: (a -> Bool) -> [a] -> [a]
filter' p = foldr
  (\x acc -> if p x then x : acc else acc) []

last' :: [a] -> a
last' = foldl1 (\_ x -> x)
```

- `reverse'`：左から順にアキュムレータの先頭にくっつける
- `product'`：アキュムレータを 1 から始め、掛け合わせていく
- `filter'`：述語を満たすときだけアキュムレータの先頭に付ける
- `last'`：常に今の要素で置き換え、最後に残ったものを返す

5.5　畳み込み、見込みアリ！

- **別の視点から見た畳み込み**

2引数関数 `f`、初期値 `z` で `[3,4,5,6]` を右畳み込み

```
f 3 (f 4 (f 5 (f 6 z)))          -- f = (+), z = 0 なら 3 + (4 + (5 + (6 + 0)))
```

2引数関数 `g`、初期値 `z` で左畳み込み

```
g (g (g (g z 3) 4) 5) 6
```

`g = flip (:)`、`z = []` なら

```
flip (:) (flip (:) (flip (:) (flip (:) [] 3) 4) 5) 6   -- [6,5,4,3]
```

5.5　畳み込み、見込みアリ！

- 無限リストを畳み込む

```haskell
and' :: [Bool] -> Bool
and' xs = foldr (&&) True xs
```

```haskell
True && (False && (True && True))
```

```haskell
(&&) :: Bool -> Bool -> Bool
True  && x = x
False && _ = False
```

```haskell
ghci> and' (repeat False)
False
```

`repeat False` を書き下すと

`False && (False && (False && ...`

最初の引数が `False` なら2つ目の引数を**評価しない**（遅延評価）

> `foldr` は、2つ目の引数を常に評価するとは限らない関数なら無限リストでも動く。 `foldl` はダメ

5.5　畳み込み、見込みアリ！

- スキャン：アキュムレータの途中経過をすべて返す

```
ghci> scanl (+) 0 [3,5,2,1]
[0,3,8,10,11]
ghci> scanr (+) 0 [3,5,2,1]
[11,8,3,1,0]
ghci> scanl1 (\acc x -> if x > acc then x else acc) [3,4,5,3,7,9,2,1]
[3,4,5,5,7,9,9,9]
ghci> scanl (flip (:)) [] [3,2,1]
[[],[3],[2,3],[1,2,3]]
```

- `scanl` は最後の要素に、`scanr` は先頭の要素に最終結果が入る
- `scanl1` / `scanr1` は `foldl1` / `foldr1` のスキャン版

5.5　畳み込み、見込みアリ！

- スキャンの例：平方根の和が 1000 を超えるのは何個目？

```
sqrtSums :: Int
sqrtSums = length (takeWhile (<1000) (scanl1 (+) (map sqrt [1..]))) + 1
```

```
ghci> sqrtSums
131
ghci> sum (map sqrt [1..131])
1005.0942035344083
ghci> sum (map sqrt [1..130])
993.6486803921487
```

- 途中経過が知りたいので、畳み込みではなくスキャン
- 和は昇順なので `filter` ではなく `takeWhile`（`filter` は走査をやめない）
- 1000 より小さい和が x 個 → x + 1 個目で 1000 を超える

HANDS-ON H4　畳み込みで書き直す（課題 問4・5）

5分

`report5.hs` の次の関数を実装し、`runghc report5.hs` で `[OK]` を確認

```haskell
map'     :: (a -> b) -> [a] -> [b]          -- foldr を使う
filter'  :: (a -> Bool) -> [a] -> [a]      -- foldr を使う
maximum' :: (Ord a) => [a] -> a            -- 第4回は再帰で実装した
reverse' :: [a] -> [a]                     -- 第4回は再帰で実装した
```

考える順番

1. アキュムレータの**型**と**初期値**は何か（空リストのときの答え）
2. 要素1つとアキュムレータから、新しいアキュムレータをどう作るか
3. 左右どちらから畳むか（リストを作るなら `foldr` + `:`）

目次

5.1　カリー化関数

5.2　高階実演

5.3　関数プログラマの道具箱

5.4　ラムダ式

5.5　畳み込み、見込みアリ！

5.6　`$` を使った関数適用

5.7　関数合成

5.6 `$` を使った関数適用

- **関数適用演算子 `$`**

```
($) :: (a -> b) -> a -> b
f $ x = f x
```

```
ghci> sum (filter (> 10) (map (*2) [2..10]))
80
ghci> sum $ filter (> 10) (map (*2) [2..10])
80
ghci> sum $ filter (> 10) $ map (*2) [2..10]
80
```

定義上はただの関数適用だが

- 空白による関数適用：**最も高い**優先順位、左結合
- `$`：**最も低い**優先順位、右結合

`sqrt 3 + 4 + 9` は `(sqrt 3) + 4 + 9`

`sqrt $ 3 + 4 + 9` は `sqrt (3 + 4 + 9)`

> `$` は「はるか右に閉じ括弧がある開き括弧」。括弧を減らして読みやすくする

5.6 `$` を使った関数適用

- **関数適用そのものを関数として扱う**

```
ghci> map ($ 3) [(4+), (10*), (^2), sqrt]
[7.0,30.0,9.0,1.7320508075688772]
```

- `($ 3)` は `$` のセクション：「関数を受け取り、その関数を 3 に適用する関数」
- リスト中のすべての関数が 3 に適用される

関数	`($ 3)` を適用	結果
`(4+)`	`(4+) 3`	7.0
`(10*)`	`(10*) 3`	30.0
`(^2)`	`(^2) 3`	9.0
`sqrt`	`sqrt 3`	1.732...

目次

5.1 カリー化関数

5.2 高階実演

5.3 関数プログラマの道具箱

5.4 ラムダ式

5.5 畳み込み、見込みアリ！

5.6 `$` を使った関数適用

5.7 関数合成

5.7　関数合成

- **関数合成演算子 `.`**

数学：$(f \circ g)(x) = f(g(x))$

```
(.) :: (b -> c) -> (a -> b) -> a -> c
f . g = \x -> f (g x)
```

`negate . (*3)` は数を受け取り、3倍して符号反転する関数

型に注意

- `g` の返り値の型 `b` = `f` の引数の型
- 合成した関数は `g` と同じ型を受け取り、`f` と同じ型を返す

```
ghci> map (\x -> negate (abs x)) [5,-3,-6,7,-3,2,-19,24]
[-5,-3,-6,-7,-3,-2,-19,-24]
ghci> map (negate . abs) [5,-3,-6,7,-3,2,-19,24]
[-5,-3,-6,-7,-3,-2,-19,-24]
```

5.7　関数合成

- たくさんの関数を合成する

関数合成は**右結合**：`f (g (z x))` は `(f . g . z) x` と同じ

```
ghci> map (\xs -> negate (sum (tail xs))) [[1..5],[3..6],[1..7]]
[-14,-15,-27]
ghci> map (negate . sum . tail) [[1..5],[3..6],[1..7]]
[-14,-15,-27]
```

`negate . sum . tail` は

1. `tail` を適用
2. その結果に `sum` を適用
3. その結果に `negate` を適用

> 関数合成は**右の関数から順に**適用する。ラムダ式を書く代わりに、その場で関数を作れる

5.7　関数合成

- **多引数関数の関数合成**

普通は、残り1引数になるまで**部分適用**しないと合成できない

```
sum (replicate 5 (max 6.7 8.9))
(sum . replicate 5) (max 6.7 8.9)
sum . replicate 5 $ max 6.7 8.9           -- 3つとも 44.5
```

- `replicate` は本来2引数だが、`replicate 5` と部分適用して1引数にしている
  → 「1つの引数を5回繰り返したリストを作る関数」
- `max 6.7 8.9` の結果が `replicate 5` に渡され、そのリストが `sum` に渡される

5.7　関数合成

- **括弧の多い式を書き直す手順**

```
replicate 2 (product (map (*3) (zipWith max [1,2] [4,5])))
```

手順	書いた式
① 一番内側の関数と引数を書き出す	`zipWith max [1,2] [4,5]`
② その前の関数を `$` でつなぐ	`map (*3) $ zipWith max [1,2] [4,5]`
③ 次の関数を `.` で合成	`product . map (*3) $ zipWith ...`
④ さらに合成	`replicate 2 . product . map (*3) $ zipWith ...`

```
replicate 2 . product . map (*3) $ zipWith max [1,2] [4,5]   -- [180,180]
```

> 式の終わりに括弧が3つ並んだら、関数合成で書き直すチャンス

5.7　関数合成

- ポイントフリースタイル

```haskell
sum' :: (Num a) => [a] -> a
sum' xs = foldl (+) 0 xs
-- 両辺の右端の xs を省略
sum' = foldl (+) 0
```

```haskell
fn x = ceiling (negate (tan (cos (max 50 x))))
-- 関数合成でポイントフリーに
fn = ceiling . negate . tan . cos . max 50
```

ポイント：`fn x = ...` の `x` のような、関数定義に現れる一時変数

ポイントを書かずに関数を定義するスタイル

`fn` の右端の `x` は括弧の中なので、そのままでは消せない。関数合成で表すと消せる

> データより関数に目が行き、「どんな関数を合成しているか」を考えやすくなる

5.7　関数合成

- 書き換えてみよう：`oddSquareSum`

```
oddSquareSum :: Integer
oddSquareSum = sum (takeWhile (<10000) (filter odd (map (^2) [1..])))
```

↓関数合成を使うと

```
oddSquareSum :: Integer
oddSquareSum = sum . takeWhile (<10000) . filter odd $ map (^2) [1..]
```

「`filter odd` が `map (^2) [1..]` に適用され、その結果に `takeWhile (<10000)`、最後に `sum`」と読める

合成のチェーンが長すぎると読みにくくなる。`let` で途中の結果に名前を付けて分割するのもよい

5.7　関数合成

- `$` と `.` の使い分け

`.` は**関数と関数**をつなぐ。`$` は**関数を値に**適用する

```
oddSquareSum = sum . takeWhile (<10000) . filter odd $ map (^2) [1..]     -- OK
oddSquareSum = sum . takeWhile (<10000) . filter odd . map (^2) $ [1..]   -- OK
oddSquareSum = sum $ takeWhile (<10000) $ filter odd $ map (^2) [1..]     -- OK
oddSquareSum = sum . takeWhile (<10000) . filter odd . map (^2) [1..]     -- ダメ
```

最後の行は `map (^2) [1..]` が先に結合して**リスト**（関数ではない）になる。
`.` の右側にリストが来るので型エラー

HANDS-ON H5　関数合成で書き換える（課題 問7）

4分

1. `report5.hs` の `oddSquareSum` を `$` と `.` を使って書く
2. 発展：`fn` をポイントフリースタイルで書く
3. GHCi で型と値を確認

```
ghci> :r
ghci> oddSquareSum
166650
ghci> :t negate . abs
ghci> :t map (*3) . filter odd      -- どんな型になる？
ghci> map fn [0, 49, 50, 51, 100]
```

まとめ

節	キーワード	ひとことで
5.1	カリー化・部分適用・セクション	引数を少なく渡すと関数が返る
5.2	`applyTwice` `zipWith` `flip`	関数を引数にとる・返す
5.3	`map` `filter` `takeWhile`	リストを変換・選別・切り出す
5.4	ラムダ式 `\x -> ...`	その場限りの無名関数
5.5	`foldl` `foldr` `scanl`	リストを1つの値にまとめる
5.6	`$`	優先順位最低の関数適用
5.7	`.` ・ポイントフリー	関数をつないで新しい関数を作る

第5回 課題

`report5.hs` を完成させて提出（型宣言は変更しない）

問	内容
問1	`zipWith'` を実装せよ（標準関数 `zipWith` は使用不可）
問2	`flip'` を実装せよ（標準関数 `flip` は使用不可）
問3	1〜100 のうち、長さが 15 より長いコラッツ列の開始数の個数を `map` と `filter` で求めよ
問4	`foldr` を使って `map'` と `filter'` を実装せよ
問5	畳み込みを使って `maximum'` と `reverse'` を実装せよ
問6	平方根の和が 1000 を超えるのは何個目か `scanl1` と `takeWhile` で求めよ
問7	`oddSquareSum` を `$` と `.` で書き換えよ（発展：`fn` をポイントフリーに）

提出前に `runghc report5.hs` で全問 `[OK]` を確認する